

Streamlit in Python (Complete Detailed Guide)

What is Streamlit?

Streamlit is an open-source Python framework used to create **interactive web applications** for Data Science, Machine Learning, AI, and Data Analysis **without using HTML, CSS, or JavaScript**.

With only Python code, you can build:

-  Dashboards
-  Data Visualization Apps
-  Machine Learning Apps
-  Forms
-  File Upload Apps
-  Chatbots
-  Image Processing Apps
-  Analytics Dashboards

Why Streamlit?

Before Streamlit, if you wanted to create a web app, you needed:

- HTML
- CSS
- JavaScript
- Flask/Django
- Bootstrap

With Streamlit, **only Python is enough**.

Example:

```
import streamlit as st

st.title("Hello Streamlit")
st.write("Welcome to Python")
```

Output

```
-----
| Hello Streamlit          |
| Welcome to Python        |
-----
```

Installation

Using pip

```
pip install streamlit
```

Check version

```
streamlit version
```

or

```
import streamlit as st

print(st.__version__)
```

Run Streamlit App

Create

```
app.py
```

Run

```
streamlit run app.py
```

It automatically opens

```
http://localhost:8501
```

Streamlit Project Structure

Project

```
|
|— app.py
|— data.csv
|— images
|   logo.png
|
|— pages
|   Home.py
|   Dashboard.py
|   Settings.py
|
└─ requirements.txt
```

Streamlit Workflow

Python Code

↓

Streamlit

↓

Browser UI

↓

User Interaction

↓

Python Executes Again

↓
Updated UI

Every interaction reruns the script.

Basic Example

```
import streamlit as st

st.title("Student Details")

name = st.text_input("Enter Name")

age = st.number_input("Age")

st.write("Name :", name)
st.write("Age :", age)
```

Streamlit Components

1. Title

```
st.title("Python Course")
```

Large Heading

2. Header

```
st.header("Machine Learning")
```

3. Subheader

```
st.subheader("Linear Regression")
```

4. Text

```
st.text("Hello")
```

5. Write

Most commonly used.

```
st.write("Hello")  
st.write(100)  
st.write([1,2,3])
```

6. Markdown

```
st.markdown("# Heading")  
  
st.markdown("***Bold**")  
  
st.markdown("*Italic*")
```

7. Code

```
st.code("""  
for i in range(5):  
    print(i)  
""")
```

8. Latex

```
st.latex(r'''  
a^2+b^2=c^2  
''')
```

User Input Widgets

Text Input

```
name = st.text_input("Enter Name")
```

Password

```
password = st.text_input(  
    "Password",  
    type="password"  
)
```

Text Area

```
feedback = st.text_area("Feedback")
```

Number Input

```
age = st.number_input("Age")
```

Slider

```
age = st.slider(  
    "Age",
```

```
1,  
100,  
25  
)
```

Select Box

```
city = st.selectbox(  
    "City",  
    ["Rajkot", "Ahmedabad", "Surat"]  
)
```

Radio Button

```
gender = st.radio(  
    "Gender",  
    ["Male", "Female"]  
)
```

Checkbox

```
agree = st.checkbox("Accept Terms")
```

Multi Select

```
skills = st.multiselect(  
    "Skills",  
    ["Python", "Java", "PHP"]  
)
```

Button

```
if st.button("Click"):  
    st.success("Clicked")
```

Toggle

```
st.toggle("Dark Mode")
```

File Upload

```
file = st.file_uploader("Upload")
```

Image only

```
file = st.file_uploader(  
    "Upload",  
    type=["jpg", "png"]  
)
```

Date Picker

```
date = st.date_input("DOB")
```

Time Picker

```
time = st.time_input("Time")
```

Color Picker

```
color = st.color_picker("Choose Color")
```

Display Images

```
st.image("cat.png")
```

Resize

```
st.image(  
    "cat.png",  
    width=300  
)
```

Display Video

```
st.video("video.mp4")
```

Display Audio

```
st.audio("song.mp3")
```

Success Message

```
st.success("Saved")
```

Error Message

```
st.error("Invalid")
```

Warning

```
st.warning("Check Again")
```

Info

```
st.info("Information")
```

Progress Bar

```
import time

bar = st.progress(0)

for i in range(100):
    time.sleep(0.05)
    bar.progress(i+1)
```

Spinner

```
import time

with st.spinner("Loading..."):
    time.sleep(3)

st.success("Done")
```

Columns

```
col1,col2 = st.columns(2)

with col1:
    st.write("Left")

with col2:
    st.write("Right")
```

Sidebar

```
st.sidebar.title("Menu")

option = st.sidebar.selectbox(
    "Choose",
    ["Home", "About"]
)
```

Expander

```
with st.expander("More Info"):
    st.write("Hidden Text")
```

Tabs

```
tab1, tab2 = st.tabs(
    ["Home", "About"]
)

with tab1:
    st.write("Home")

with tab2:
    st.write("About")
```

Metrics

```
st.metric(
    "Revenue",
    "₹50,000",
    "+10%"
)
```

Dataframe

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["A", "B"],
    "Age": [20, 25]
})

st.dataframe(df)
```

Table

```
st.table(df)
```

Charts

Line Chart

```
import pandas as pd
import numpy as np

chart = pd.DataFrame(
    np.random.randn(20, 3),
    columns=["A", "B", "C"]
)

st.line_chart(chart)
```

Bar Chart

```
st.bar_chart(chart)
```

Area Chart

```
st.area_chart(chart)
```

Forms

```
with st.form("login"):

    name = st.text_input("Name")

    password = st.text_input(
        "Password",
        type="password"
    )

    submit = st.form_submit_button("Login")

if submit:
    st.success("Logged In")
```

Session State

Stores values between reruns.

```
import streamlit as st

if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Increase"):
    st.session_state.count += 1

st.write(st.session_state.count)
```

Caching

Avoids repeated expensive computations.

```
@st.cache_data
def load_data():
    return pd.read_csv("data.csv")
```

Download Button

```
st.download_button(
    "Download",
    data="Hello",
    file_name="hello.txt"
)
```

Camera Input

```
image = st.camera_input("Take Photo")
```

Chat Interface

```
prompt = st.chat_input("Ask")

if prompt:
    st.chat_message("user").write(prompt)
```

Secrets

Create

```
.streamlit/secrets.toml
```

```
API_KEY="12345"
```

Read

```
st.secrets["API_KEY"]
```

Configuration

Create

```
.streamlit/config.toml
```

Example

```
[theme]

primaryColor="#FF4B4B"

backgroundColor="#FFFFFF"

secondaryBackgroundColor="#F0F2F6"

textColor="#000000"
```

Multipage App

```
project
|
├─ app.py
|
└─ pages
    Home.py
    About.py
```

Streamlit automatically creates navigation.

Real World Projects

Beginner

- Calculator
- Unit Converter
- BMI Calculator
- Student Result System
- To-Do App

Intermediate

- Weather Dashboard
- Expense Tracker
- Notes App
- File Converter
- PDF Merger

Advanced

- ChatGPT Clone
 - Resume Analyzer
 - Stock Market Dashboard
 - Face Recognition App
 - Object Detection App
 - AI Image Generator
 - Document Q&A using LLMs
-

Advantages

- Very easy to learn
 - Pure Python
 - No frontend knowledge required
 - Fast development
 - Great for Machine Learning
 - Interactive widgets
 - Automatic rerun
 - Open source
 - Multipage support
 - Easy deployment
-

Disadvantages

- Less customizable than traditional frontend frameworks
 - Not ideal for very large enterprise web applications
 - Every interaction reruns the script (requires understanding of session state)
 - Limited control over low-level HTML/CSS compared to custom web apps
-

Streamlit vs Flask

Feature	Streamlit	Flask
Purpose	Data apps, dashboards, ML apps	General web development
Frontend	Built-in	HTML/CSS/JS required
Learning	Easy	Moderate
Routing	Multipage support	Manual routing

API Support	Limited	Excellent
Database	Supported	Supported
ML Integration	Excellent	Good
Deployment	Simple	More flexible
Best For	Data science, analytics, AI demos	Full-stack web applications

When to Use Streamlit

Use Streamlit when you want to:

- Build dashboards quickly.
- Create machine learning model demos.
- Visualize data interactively.
- Prototype applications rapidly.
- Build internal tools for data teams.

Choose Flask (or similar frameworks) when you need:

- Authentication and authorization systems.
 - Complex URL routing and REST APIs.
 - Full e-commerce or social networking websites.
 - Fine-grained control over the frontend.
-

Interview Questions

1. What is Streamlit?
2. Why is Streamlit popular for data science?
3. How do you install Streamlit?
4. How do you run a Streamlit application?
5. What is `st.write()` ?

6. What is `st.session_state` ?
 7. What is the difference between `st.cache_data` and `st.session_state` ?
 8. How do you create a sidebar?
 9. How do you upload a file in Streamlit?
 10. How do you create a multipage Streamlit application?
 11. What is the purpose of `st.form()` ?
 12. How can you deploy a Streamlit app?
-

Learning Path

1. Install Streamlit and create your first app.
2. Learn text elements (`st.title` , `st.write` , `st.markdown`).
3. Practice input widgets (text boxes, sliders, buttons).
4. Display data using `pandas` DataFrames and charts.
5. Learn layouts (columns, tabs, sidebar, expanders).
6. Understand forms, session state, and caching.
7. Build a multipage project.
8. Integrate machine learning models and deploy the application.

Since you've been learning **Python, Flask, and Data Science**, Streamlit is an excellent next step. It lets you turn your Python scripts and ML models into interactive web apps with very little code, making it ideal for showcasing projects on GitHub and in your portfolio.